Reflexión técnica: IA como herramienta en el desarrollo de un analizador de logs

- **Aportes de la IA**

  La IA propuso una estructura modular con funciones para analizar una línea, analizar el archivo, seleccionar el archivo y presentar resultados. También sugirió una expresión regular para reconocer severidades al inicio de la línea, y el uso de `datetime.date.fromisoformat` para comprobar que una fecha `YYYY-MM-DD` existe en el calendario y no solo tiene la forma correcta.

  Por ejemplo, la validación de fecha combina una comprobación de formato con la validación del calendario:

```
if not ISO_DATE_PATTERN.fullmatch(value):
    return False

try:
    date.fromisoformat(value)
except ValueError:
    return False
return True
```

  La IA también implementó el selector gráfico con `tkinter` , el conteo por severidad y la presentación del resumen. Posteriormente, a petición del estudiante, incorporó la pregunta para mostrar el detalle, el formato multilínea con tabulación, los colores de severidad y la omisión de líneas vacías. Estos cambios técnicos fueron implementaciones de requisitos revisados con el estudiante; no determinan por sí solos qué datos deben considerarse inválidos.

- **Decisiones del estudiante**

  Realicé ajustes y definí los criterios principales del programa:

  1. Reconocer “INFO” , “WARNING” y “ERROR” solo cuando aparecen al inicio con corchetes, por ejemplo, “[INFO]”.
  2. Aceptar como fecha válida únicamente una fecha real en formato ISO `YYYY-MM-DD` .
  3. Contar como evento toda línea con severidad reconocida, aunque le falte una fecha válida o un mensaje.
  4. Marcar como incompleta una línea con contenido si no tiene severidad reconocida, fecha válida o mensaje.
  5. Ignorar las líneas vacías o que contienen solo espacios y sí evaluar las líneas con contenido, aunque estén incompletas.
  6. Mostrar primero el resumen y preguntar si se desea ver el detalle línea por línea.

7. Presentar cada detalle en renglones separados y tabulados, con las etiquetas [INFO]: azul, [WARNING]: amarilla y [ERROR]: roja, cuando la consola admite color.
8. Conservar la numeración original del archivo en el detalle, aunque haya líneas vacías ignoradas.

También se definí como manejar la cancelación del selector y los problemas al abrir o leer el archivo con mensajes claros. Se cubrieron casos bordes como fechas inexistentes (`2025-02-30`), formato de fecha no admitido, severidad desconocida, mensaje ausente y líneas vacías.

- **Ejemplo de propuesta mejorable y corrección**

  En la versión de prueba en la rama 'Analizador-experimental', el detalle se imprimía antes del resumen, en una sola línea por registro, y siempre se mostraba. Por ejemplo, el formato inicial reunía nivel, fecha, mensaje y estado en una línea:

  Línea 1: nivel=INFO, fecha=válida (2025-01-10), mensaje="User logged in", estado=completa

  Esto era funcional, pero mejorable para la interfaz solicitada. El resumen quedaba al final, el usuario no podía omitir el detalle y las categorías no resaltaban visualmente. Además, `analyze_lines` evaluaba cada renglón leído sin filtrar los vacíos, por lo que un renglón en blanco se contaba como incompleto.

  Al detectar esos aspectos, solicite primero cambiar el orden y hacer opcional el detalle, luego especifiqué el formato multilínea y los colores. Por último, aclare que los renglones vacíos debían ignorarse, pero que las líneas con contenido incompleto sí debían evaluarse. La versión de la rama 'main' aplica ese filtro antes de analizar y guarda el número original de línea para mostrarlo correctamente:

```
for line_number, line in enumerate(lines, start=1):
    if not line.strip():
        continue
    result = analyze_line(line)
    result.line_number = line_number
    results.append(result)
```

  Así, una línea vacía no suma como error ni aparece en el detalle, mientras una línea con contenido, pero sin fecha válida continúa contándose como incompleta. El detalle solo se imprime cuando la respuesta a la pregunta es afirmativa.

- **Aprendizaje**

  Este trabajo muestra que la IA puede acelerar la creación de una estructura inicial, proponer validaciones y ayudar a implementar cambios repetitivos. Sin embargo, una primera versión plausible no garantiza que el comportamiento coincida con la consigna ni con la experiencia que necesita el usuario. La versión inicial trataba un renglón vacío

como incompleto y la interfaz no ofrecía elegir si se quería ver el detalle, ambos comportamientos se corrigieron al revisar el resultado y precisar los requisitos.

Como desarrolladora mantuve el control sobre las reglas, detecté oportunidades de mejora y pedí cambios concretos. También verifique los casos con líneas vacías, fechas válidas e inválidas, severidades desconocidas, y las dos respuestas posibles a la pregunta del detalle.

En conclusión, la IA funciona mejor como apoyo para proponer e implementar. La definición de criterios, la evaluación de resultados y la aceptación final del programa siguen siendo responsabilidad humana.